

SIMATS SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI - 602105

Assignment Answer Booklet

Requirements Engineering and Software Architecture – UNIT II

Name	:	Musahith Arafath S
Register No.	:	192524400
Course Code	:	CSA1006
Course Name	:	Software Engineering
Unit	:	Unit II – Requirements Engineering and Software Architecture
Question Attempted	:	Problem 3 – Choosing Architecture for a Scalable Food Delivery Platform

Scenario Recap

A startup wants to build a food delivery platform, similar to Swiggy or Zomato, that must scale during peak hours and process thousands of orders per minute. The CTO is unsure whether to use microservices, serverless, or a monolithic architecture for the system.

Task

Recommend an architecture, justify the decision, explain the role of event-driven architecture, and list suitable microservices.

1. Recommended Architecture

For a food delivery platform expected to process thousands of orders per minute with sharp, unpredictable peaks (lunch and dinner rushes, festival days, weather-driven demand spikes), the recommended architecture is a **microservices architecture built on an event-driven backbone**, with selected latency-sensitive or bursty components deployed as serverless functions. A pure monolith is not recommended, and pure serverless-for-everything is also not recommended; the hybrid combination best matches the platform's mixed workload characteristics.

In this architecture, core domains such as Orders, Delivery, Restaurant Catalog, and Payments are implemented as independently deployable microservices, each owning its own data store. These services communicate primarily through an event bus (such as Kafka or a managed equivalent) for state changes that other services need to react to, and through synchronous REST/gRPC APIs only where an immediate response is required, such as an order placement confirmation.

Architecture Options Considered

Option	Suitability for This Platform
Monolithic	Simple to build initially, but a single spike in order volume (e.g., a lunch rush) forces the entire system to scale, leading to high costs and potential performance degradation.
Pure Serverless	Scales automatically and removes server management, but cold starts add latency that hurts user experience for time-sensitive actions.
Microservices + Event-Driven (Recommended)	Each domain (orders, delivery, payments) scales independently based on its own demand, providing a hybrid of scalability and performance.

2. Three Technical Reasons for the Choice

1. Independent, Demand-Matched Scaling

Order volume, restaurant catalog browsing, and delivery-partner location updates each follow very different traffic curves. With microservices, the Orders service can be scaled to 50 instances during a dinner rush while the Restaurant Catalog service, which changes far less often, stays at a handful of instances. A monolith would force the whole application to scale as one unit, multiplying infrastructure cost.

2. Fault Isolation and Resilience

If the Payments service experiences a slowdown from an external gateway, an event-driven microservices design allows Order placement to continue accepting orders and queue payment events for processing once the gateway recovers, rather than the entire application going down. This directly supports the platform's requirement to keep taking orders during peak hours even if one dependency is degraded.

3. Independent Deployability and Faster Iteration

Separate teams can own the Delivery, Orders, and Payments services and deploy changes to each without coordinating a full-application release. For a startup that needs to iterate quickly on features like live delivery tracking or dynamic pricing, this significantly reduces release risk and time-to-market compared with redeploying a single monolithic codebase.

3. Role of Event-Driven Architecture

Event-driven architecture (EDA) is the mechanism that lets the microservices above work together without becoming tightly coupled. Instead of the Orders service directly calling the Delivery service and waiting for a response, the Orders service publishes an **OrderPlaced** event to a message broker. The Delivery service, Notification service, and Analytics service each subscribe to that event independently and react in their own time.

Key Benefits for a Peak-Load Food Delivery Platform

- **Load levelling:** during a sudden spike, events queue in the broker instead of overwhelming downstream services, smoothing out bursts rather than dropping requests.
- **Loose coupling:** new consumers (for example, a future fraud-detection service) can subscribe to existing events like **PaymentCompleted** without any change to the Payments service itself.
- **Real-time responsiveness:** customer-facing features such as live order-status tracking and delivery-partner location updates are naturally modeled as streams of events rather than repeated polling of a database.
- **Resilience through replay:** if the Notification service goes down temporarily, it can replay missed events from the broker's log once it recovers, instead of losing those notifications entirely.

Representative Event Flow for Order Placement

Step	Event / Action	Consuming Service(s)
1	OrderPlaced published by Orders service	Delivery, Notification, Analytics
2	DeliveryPartnerAssigned published by Delivery service	Notification, Orders
3	PaymentCompleted published by Payments service	Orders, Analytics

4	OrderDelivered published by Delivery service	Notification, Analytics, Ratings
---	--	----------------------------------

4. Suitable Microservices for the Platform

The platform is decomposed along clear business-capability boundaries so that each service has a single, well-defined responsibility and its own data store, following the database-per-service pattern.

Microservice	Responsibility
Orders Service	Creates and tracks orders, manages order state transitions (placed, confirmed, preparing, out for delivery, delivered, etc.)
Delivery Service	Matches available delivery partners to orders, tracks live location, and calculates estimated time of arrival
Payments Service	Integrates with payment gateways, handles refunds, and publishes payment status events
Restaurant Catalog Service	Manages restaurant profiles, menus, pricing, and availability, optimized for high-read, low-write
Notification Service	Consumes events from Orders, Delivery, and Payments to send SMS, push, and email notifications
Ratings and Reviews Service	Collects post-delivery ratings and feedback and feeds an aggregate score back to the Restaurant Catalog

The task explicitly requires at least three microservices (for example, Orders, Delivery, and Payments); the table above extends this to six to reflect a realistic minimum viable decomposition, while keeping each service's responsibility narrow enough to scale and deploy independently, consistent with the architecture recommended in Section 1.

5. Summary

- A hybrid microservices plus event-driven architecture was recommended over a pure monolith or pure serverless approach, based on the platform's uneven, bursty order traffic.
- Three technical reasons were justified: independent demand-matched scaling, fault isolation, and independent deployability.
- Event-driven architecture was explained as the coupling mechanism between services, with a sample order-placement event flow across four services.
- Six microservices were proposed — Orders, Delivery, Payments, Restaurant Catalog, Notification, and Ratings — each owning a distinct business capability and data store.